

FOSS IS *fun* Free Your Mind

Golf, par, handicaps—and sub-classing, abstraction and lines of code. Sounds interesting?

Last month, I have had the privilege of listening to two presentations on two radically different subjects. The only thing in common between them was that both presenters were masters in their respective fields—guys who ‘wrote the book’. Both were Americans, and strangely enough, although the subject matter of their respective presentations had absolutely nothing to do with each other, the message both conveyed was exactly the same.

The first presentation was a three-day workshop on golf course rating, conducted by Eric Lahman of the USGA (United States Golf Association) at the Bangalore Golf Club. The second was a talk on ‘The art of sub-classing’, at the annual Indian Python Conference, by Raymond Hettinger, a core Python developer and PSF (Python Software Foundation) board member.

The concept of golf is very simple. As P G Wodehouse once said: ‘You hit a ball with a stick till it goes into the hole,’ and whoever hits the ball the least number of times, over 18 holes, wins. Over the past 100 or more years, the concept of ‘par’ has become synonymous with golf. A par-3 hole is one where the golfer is expected to ‘hole out’ in 3 shots. Similar to the case of par-4 and par-5 holes. The par for each hole is fixed by a committee that runs the particular course. However, the golfer in question may not be capable of doing a par-3 in 3 strokes. The USGA has an elaborate system of rating courses, and so each golfer is entitled to subtract a certain number of strokes from his score to enable him to compete; this is called his handicap.

As a golfer, I need to analyse each hole to see how many strokes I need, at my level, to complete the hole. If I aim at this target, I have a good chance of winning. But I am conditioned to try for the impossible ‘par’,

and get disheartened and lose. Par is irrelevant—heresy, but it works. Eric is one of the people developing the USGA system, so this comes from the horse’s mouth.

Raymond, on sub-classing, had the same message. The parent class controls all its subclasses? Wrong; the subclass controls the parent. When inheriting, should you choose the most abstract of all classes as the parent, and build your subclasses around that? You will get full marks in any exam you write—but otherwise, this is useless.

Why would you want to use inheritance anyway? Because it is cool? Because it is the way? Because the whole idea of object-oriented programming is to replicate the natural order of things? Before you answer this, let me ask you another question: what do you aim at in writing code?

I am not talking of the proprietary world, here—the code coolies who write code for money—the more lines, the more money. We, on the other hand, want to achieve the maximum results with the minimum code. So when we choose the parent class, we choose that object with the maximum amount of re-usable code—and when we play golf, we ignore par and shoot to win.

BTW, if you meet a golfer after his round and he says he was ‘just playing for exercise’, it means he lost. And if you meet a programmer who boasts about the number of lines of code he can write in a day, don’t hire him.

Interested readers can check out the details of Raymond’s talk on the in.pycon.org website. 

By: Kenneth Gonsalves

Kenneth Gonsalves is a FOSS activist and consultant based in Ooty, and he blogs at <http://lawgon.livejournal.com>.